

Progress Description & Overview

The overall goal of my project was to expand on the ray tracer from project 3 to be able to render objects represented by Signed Distance Fields (SDFs) in order to morph objects and create morphing animations. I was inspired by the type of image manipulation and facial feature interpolation which was used in the music video for the song "Black or White" by Michael Jackson. The difference here is that this kind of morphing involves two existing images, and interpolation between them. The morphed image is entirely dependent on the two starting images and cannot be arbitrarily placed in 3D space. With these limitations in mind, my idea for this project was to be able to represent an object that is "half-way" between one object and another in a way that is computationally tangible, that is, this half and half object can be placed in a scene under different lighting conditions and be given specific colors. Using SDFs, I was able to achieve this, with limitations. My project can render certain SDF primitives, spheres, boxes, and tori. However, part of my goal that was not achieved was creating a way to get a sdf function of an object composed of many triangles.

One of my biggest obstacles when it came to implementing SDFs was the raymarching process when it came to refracting the view ray through objects. Signed distance fields are called "signed" because they return a negative value when inside an object and a positive value when outside the object. When you are trying to march along a ray by the distance to an object, this can get wonky when the distance function returns a negative number. For refraction previously, knowing whether the ray was inside an object only mattered when the ray hit an object. So, my existing check for whether we were exiting or entering an object using the direction of the surface normal would not work here. Realizing that this was my issue was a long process. Another headache was getting rotation to work on sdf objects. My ray tracer was built on the projective geometric algebra backbone, which comes with a handy tool, motors. Seems simple to add a motor to the distance function, reducing all distance calculations to local space. Easier said than done. I spent hours working to figure out why my transformations of points were coming out as NaNs, then why things would only render at the origin, then why my objects would render at the wrong location when they were at an angle. Along the way, I realized that my multiplication function for multivectors was missing pieces and had to be very careful about the way I constructed and layered my motors. Some of my issues still remain unsolved, so rotation can cause the location of objects to be slightly different than as intended.

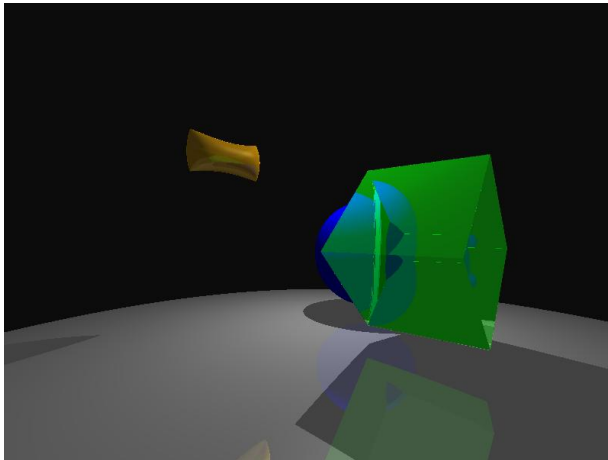
Connection to Our Class

This project directly related to the rendering pipeline and lighting model aspects of our class which we learned about to create our ray tracer for project 3. This project uses much of the ray tracer backbone built in project 3 and expands on it by implementing the rendering of signed distance fields using signed distance functions. The use of signed distance functions allows for a form of optimization in the ray tracer via raymarching. Normally when we cast a ray from the eye to a pixel, the ray tracer needs to calculate whether it intersects an object. Ray-sphere intersection math is realistically doable, however when it comes to more complex shapes, like tori, or even fractals, that math can become much more difficult. Often times it is much less computationally heavy to be able to calculate the distance to an object. So, with raymarching, when we cast a ray, we calculate the distance to all of the objects in the scene,

move along by the minimum distance, and repeat until the minimum distance is so small that we must be touching an object. To get the normal at that point we can use the distance function at very close points to estimate the gradient at that point. In this way, calculating whether we hit an object and the normal at the point is the same for all signed distance field objects. So, this project utilizes a change to the rendering pipeline that offers an optimization to rendering certain objects.

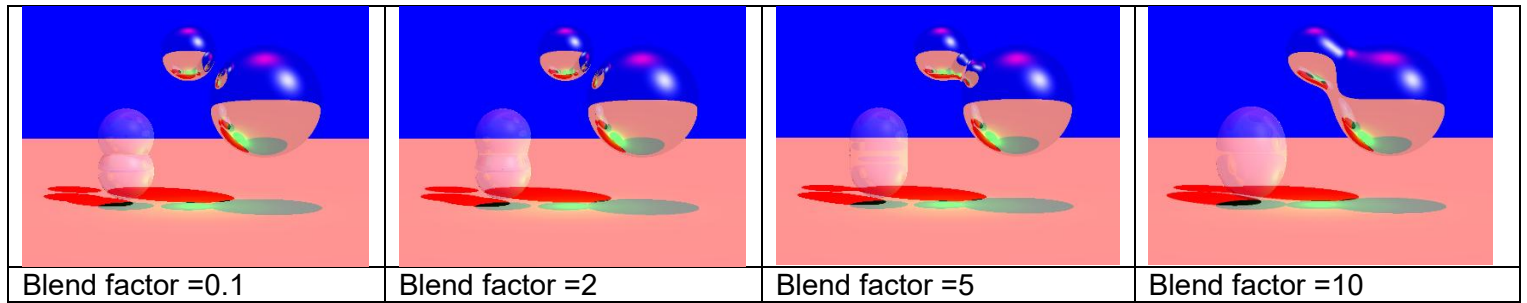
This project also displays a small amount of time-based animation. I wrote a few animation creation programs, one for each animation. Each program iterates over the total frames and uses the intended frame rate to calculate the current time at that frame. It then writes to a temporary scene file, updating time-based parameters, uses the scene file parser and raytracing functions to generate the frame image, and deletes the temporary scene file. At the end, I used ffmpeg to stitch together the frames to make videos.

Key Features



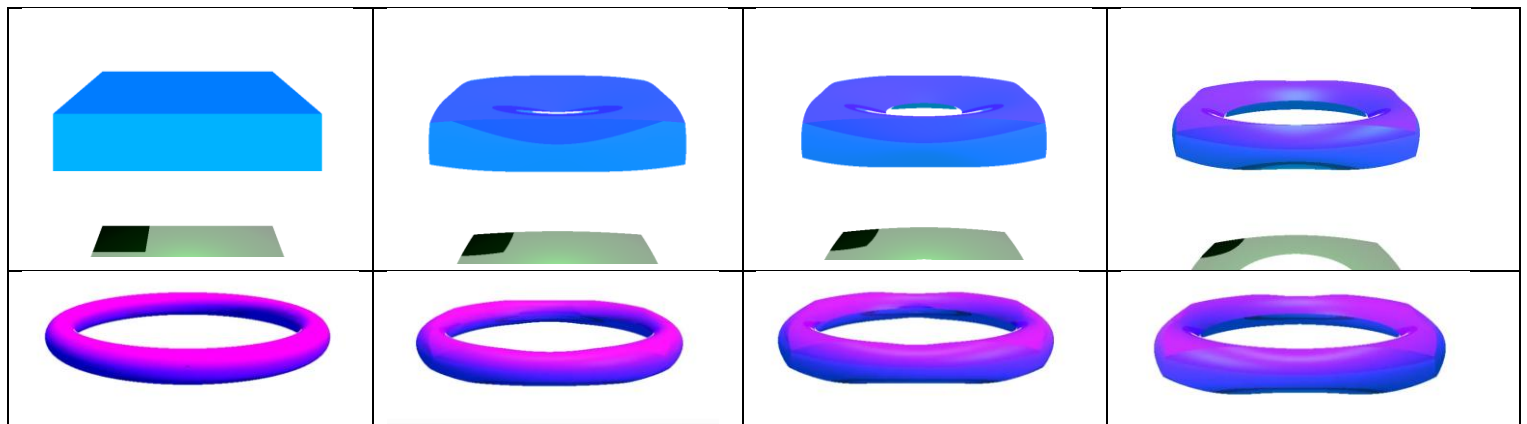
1 A box on the right with a sphere behind it, and a morph at $t = 0.5$ between two semi-overlapping tori.

The key features of this project are the implementation of sdf spheres, boxes, tori, smooth union, and morph. All objects can vary in position and material. SDF spheres render the same as regular spheres in the original ray tracer and can vary in radius, boxes are rendered as solid objects and can vary in angle and the lengths of their sides, and tori are rendered as solid objects and can vary in angle, the radius affecting the size of the ring, and the radius affecting the thickness of the ring. Union and morph objects are created by first defining two sdf objects. For union objects, the blend factor can be varied to create more of a smoothing effect between them. A blend factor near 0 results in a hard union, while a higher blend factor creates more blending between objects and bridging of the space between non-overlapping objects.



I experimented with interpolating the color of each hit point based on the distance from the hit point to each of the objects, which can be seen in bounce.mp4. However, this created weird issues when the union objects were tori, so this part of the code is commented out, and instead union objects use the color of object A.

For morph objects, the time of the morph function can be varied, where at time 0 the morph is identical to object A, at time 1 it is identical to object B, and times between 0 and 1 show the morphing of one object into the other. This is done by interpolating between the distance to object A and the distance to object B with time as the interpolation factor. The color of the morph object is also interpolated between the colors of object A and object B.



Resources Used

This project was built off of the original ray tracer for this class, which uses the `stb_image`, `multivector`, and `3D PGA` classes given to us by Stephen Guy. It also uses `std` libraries for reading and writing text files, interpreting command line inputs, and math.

For an introduction to SDFs and raymarching, I watched Sebastian Lauge's Coding Adventure: Raymarching. I also found the resources listed in the description of the video to be enlightening reads. Most notably, I found Inigo Quilez's articles on SDF primitives and smooth minimum functions to be very useful in designing my sdf class.

I went into this class without any experience using a coding language that is specific about variable storage and accessing the way that C++ is, so Gemini AI was a huge help with understanding how to use this aspect of the language to my advantage. I did not know that passing variables by reference was something I could do to speed up my code until it was suggested to me by Gemini. There were also many times when my only partial understanding of the C++ syntax led to unforeseen logic errors that I could not look up the way that I could a syntax error. For example, the way I had written my SDF class as a child of my hittable class from my original ray tracer was creating new `SDFObj` specific color variables, and so when my original ray tracer tried accessing them it would use the hittable class variables and not color the object. This was a logic error I would have never figured out without either having Gemini or someone familiar with the language look through my project files, the latter being unavailable while I am away from campus over the summer.

Future Work

There are a lot of ideas I had for this project that would be great steps for the future. I would love to be able to better flush out the animation part of the project. I had originally started trying to develop an animation scene file parser, spending a good deal of time working out different commands to set certain variables to depend on time. For example, instead of the command `"sdf_sphere:"` being followed by four floats, it could be followed by different variable names, which a separate command would set the function of time that the variable depends on. Finishing the animation parser would be a reasonable next step.

Another next step would be to create sdf functions for objects composed of vertices and triangles, which follows my original idea for the project. Additionally, there are some SDF functions that define fractals which would be interesting to try to render. I suspect that my project at its current state is not optimized enough to render these, so it may be something that would first require the implementation of something like a bounding volume hierarchy or parallel computing to do. Something else that could be added is the ability for animation variables to interact with each other. This could be used in something like a gravity bouncy ball situation, where the position of the radii of two spheres being at radius to radius distance triggers a collision event which changes the direction of the points representing the centers of the objects. With SDFs, this could be expanded to a fluid simulation where the density of particles affects the distance function to a "fluid" object represented by a type of union of the particle objects.